计算机学院 高级语言程序设计 课程实验报告

实验题目：魔兽世界大作业		完成难度：难度三（100%）
学号：202200130077	姓名：薛润泽	班级：22 级人工智能

程序文件的组织架构：

程序文件共两个文件，头文件”warrior.h”和源文件”魔兽世界 main.cpp”。

头文件”warrior.h”中包含多个武士类。武士类包括 dragon 类，ninjia 类，iceman 类，lion 类和 wolf 类，它们由同一个类 “warrior”类派生。”warrior”类中包含了各类武士都需要的数据和函数，例如使用武器的函数。如果派生类武士有特殊的函数使用要求，例如：ninjia 在使用 bomb 时自身不受到伤害，那么使用 bomb 的函数要在 warrir 类中以虚函数的形式声明，并且在 ninjia 类中重写该函数。该头文件中还包含了武士之间战斗、抢夺、武器获得等多个函数。

源文件”魔兽世界 main.cpp”中含有生成红蓝双方武士的 createredwarrior()函数，createbluewarrior()函数，和 main()主函数。其中 main()主函数的主要功能是实现时间的推进，主函数中主要使用了 while()来进行循环，从而使两方司令部不断进行产生武士、武士前进、武士战斗、报告战斗情况，直到时间结束或者一方司令部被占领。

程序文件的实现逻辑：

程序主要实现利用了多态性来实现。dragon 类,ninjia 类,icaman 类,lion 类和 wolf 类都是由 warrior 类派生的子类。父类 warrior 中含有子类中所有函数的虚函数，便于在主函数中通过使用 warrior 类的指针来调用武士类的派生类以及相应函数。

1. 五个武士类 + 一个武士基类的构建

（1）武士基类 warrior 类实现逻辑：

warrior 类与其他派生类中数据和函数都是 public，省去了接口的编码，便于直接操作数据和函数。

warrior 的基本数据有：

credit:记录是什么派生类武士,credit==1 是 dragon 以此类推,2 是 ninjia,3 是 iceman,4 是 lion，5 是 wolf;

aim:检测是否到达对方司令部;

color:分辨该武士是红魔军还是蓝魔军,color==0 是红，color==1 是蓝

city:记录武士走到哪一个城市了（用的较少，主程序中直接输出了所在城市）

number：记录武士编号

HP:武士的生命值

aggressivity:武士的攻击力

weapon[3][2]:存储武士的武器，其中值为武器的数目,weapon[0][1]存储 sword,weapon[1][1]存储 bomb,weapon[2][0]存储耐久为 1 的 arrow,weapon[2][1]存储耐久为 2 的 arrow

keeparrow 暂时保存耐久为 2，但是已经被使用耐久降为 1 的 arrow

weapnsum 记录总武器的数量，但是不包括 keeparrow 中未更新的 arrow

symfight 检测是否参与了最近一次的战斗，0 则没参与，1 则参与

warrior 中函数

void gethurt(long long hurt) 对该武士造成伤害

long long usesword()武士使用 sword，返回 sword 造成的伤害

virtual long long usebomb()武士使用 bomb 造成伤害，因为 ninjia 特性，需要写为虚函数便于多态性的实现

virtual void cheer(long long time)dragon 武士在战斗结束后欢呼(实际上主要在主函数中实现)

virtual void icemanwalk()iceman 武士在移动过程中损失 HP

virtual void changeloyal(int K) lion 武士在移动过程中损失 loyal

virtual void wolfrobweapon(warrior* dr, long long recordtime) wolf 武士在战斗前偷到对方的武器

virtual void getweapon(warrior* dr)所有在战斗后胜利的武士，取走对面武士的武器作为战利品

long long usearrow0() 使用耐久为 1 的 arrow

long long usearrow1() 使用耐久为 2 的 arrow

void updatearrow()更新 keeparrow 到 arrow[2][0]中，并将 updatearrow 归零

void createweapon(int n)用于武士出生时获取相应武器

long long checkweaponsum() 计算所有武器数目(包括 keeparrow)并返回武器数目

void countweapon()用于在战斗时给武器排序

virtual int getloyal()返回该 lion 武士的忠诚值

virtual void reportsituation()报告武士所处的阵营和职业

（2）dragon 类实现逻辑

公有继承 warrior 类，使用类的构造函数给相应数据赋值，并且重写 void reportsituation()函数实现多态(五个派生类都重写了该函数)；

（3）ninjia 类实现逻辑

公有继承 warrior 类，构造函数赋值数据,重写了 long long usebomb()函数

(4)iceman 类实现逻辑

公有继承 warrior 类，构造函数赋值数据,重写了 void icemanwalk()函数

(5)lion 类实现逻辑

公有继承 warrior 类，定义一个 public 数据 loyal,构造函数赋值数据,重写了 void changeloyal(int K),int getloyal()等函数

(6)wolf 类实现逻辑

公有继承 warrior 类，构造函数赋值数据,重写了 wolfrobweapon(warrior*dr,long long recordtime)函数

2. 部分函数实现逻辑：

（1）void battlefight(warrior*A, warrior*B, int checkparity)函数用来实现红魔蓝魔双方两个武士的战斗。调用函数时，向函数中传入两个武士的指针，在函数体中循环调用两个武士进行互相攻击。

（2）void outcitysituation(warrior*A,warrior*B,long long recordtime,int citynum)函数实现报告战斗状态，通过检测传入的 A,B 指针代表的武士的 HP 状态输出相应的战况

（3）warrior *createredarmy()与 warrior *createbluearmy()用于生产武士，并且返回该武士的地址，在生产武士时按照颜色，序号，种类，HP，攻击力等一一给新生武士赋值（使用构造函数）

3. 主函数实现逻辑：

主函数通过两层循环实现了代码的运作。第一层是 for 循环，从 1 循环到 t 实现了多组数据的输入。第二层循环是 while 循环。使用 warrior*类型的二维数组 citysets[][]来保存武士，第一维表示城市，第二维表示武士颜色（0 为红色，1 为蓝色），值表示武士的指针。特别的

citysets[0][1]表示蓝魔武士攻入红方司令部，citysets[n+1][0]表示红魔武士攻入蓝方司令部。在 while 循环一开始进行武士的创建。之后进行两军的移动，判断是否占领对方司令部。输出所有移动状态、健康状态和攻击力。wolf 先抢夺武器，再开启对战，对战结束后，胜利方缴获死亡方的武器。报告战斗结果（我在报告战斗结果结束后归零了死亡武士并且释放相应堆）。此后不断进行循环，直到一方司令部被占领或者达到（超过）指定时间。

